CSCI 580 Assignment 4: Simulated Annealing and Genetic Algorithm for TSP

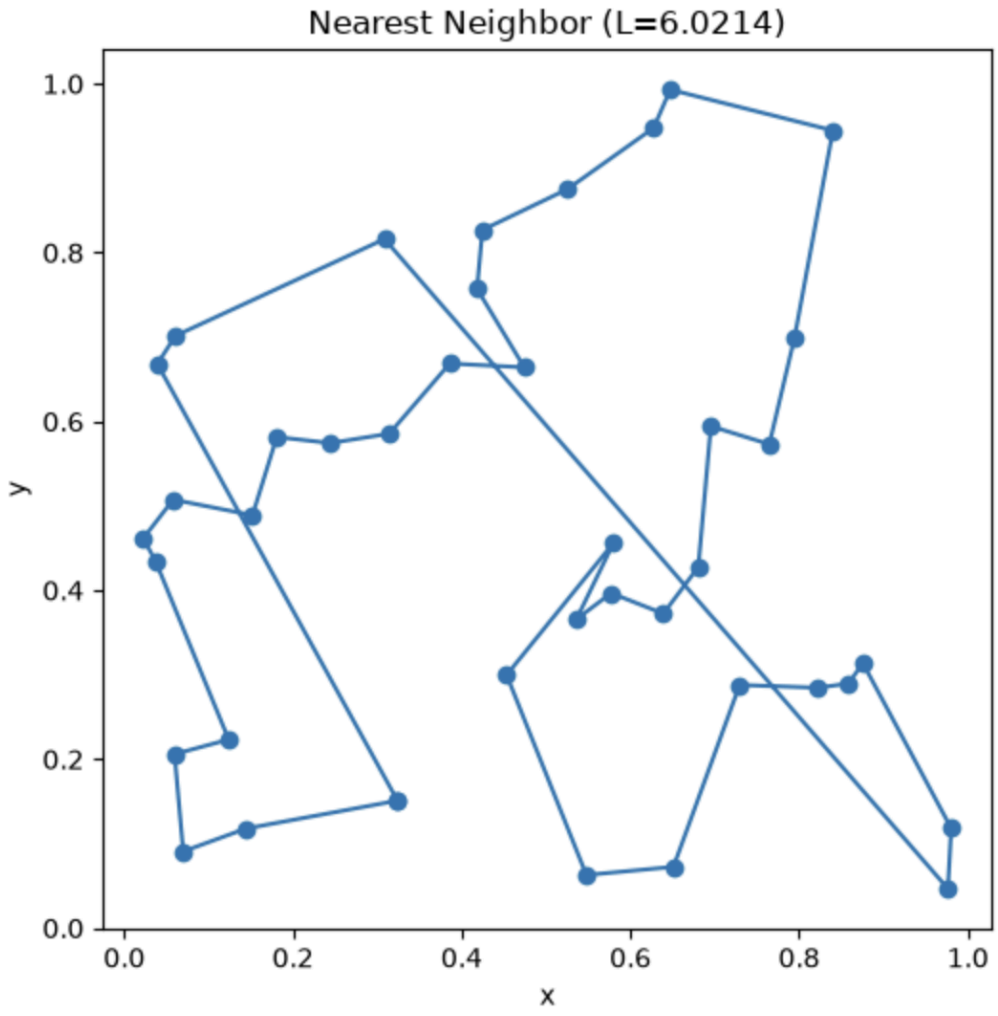
Lauren Runion

1. Problem

The problem is The Traveling Salesman Problem (TSP) which is about finding the shortest closed tour that visits every single city exactly one time. So in this assignment I used 40 random cities that were in a 1x1 square(seed). For a baseline I used the nearest neighbor method, which is just a method that goes to the closest unvisited city every time. This gave a tour length of 6.0214, I then tried to beat that using Simulated Annealing and a Genetic Algorithm.

Look at nearest neighbor tour plot

```
Nearest-neighbor length: 6.0214
```



2. Simulated Annealing

Simulated Annealing starts with the nearest neighbor tour and then picks a random 2 opt move. This happens each iteration which ends up reversing a section of the tour. If the new tour is shorter, then it is always accepted. If the new tour is longer, it is accepted with probability exp(-delta / T), T is temp. T then gets multiplied by 0.9995 each iteration so it slowly cools which lets SA accept bad moves early on and settle down later.

```
if delta <= 0:
    accept = True
else:
    accept = rng.random() < math.exp(-delta / T)

if accept:
    accepted_deltas.append(delta)
    cur_tour, cur_len = cand_tour, cand_len
else:
    rejected_deltas.append(delta)

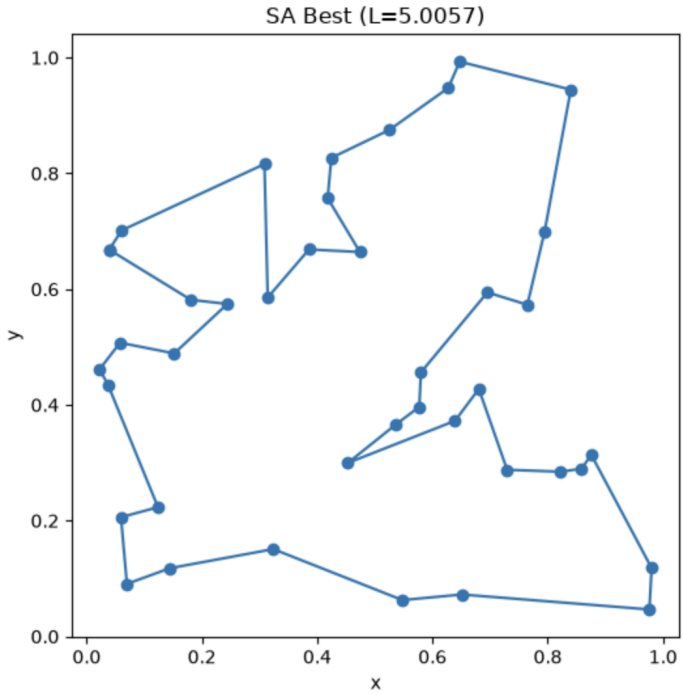
if cur_len < best_len:
    best_tour, best_len = cur_tour[:], cur_len
```

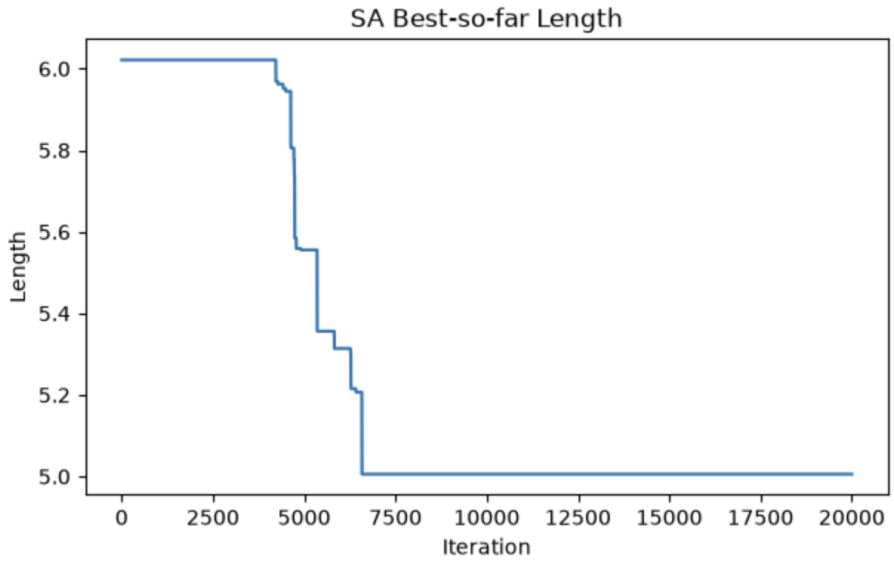
```
    # now is T adjusted?

    T = max(T * cfg.alpha, 1e-12)
```

Parameters: 20,000 iterations, start temp 0.2, cooling 0.9995, seed 123.
Best tour length: 5.0057 (16.87% better than nearest neighbor).

```
SA best length: 5.0057
SA improvement vs NN: 16.87%
```





3. Genetic Algorithm

The Genetic Algorithm keeps a population of tours and each generation, it will pick parents using tournament selection (best of 3 random tours). It will then make a child with order crossover (OX), and then sometimes apply swap mutation. The two best tours will always be

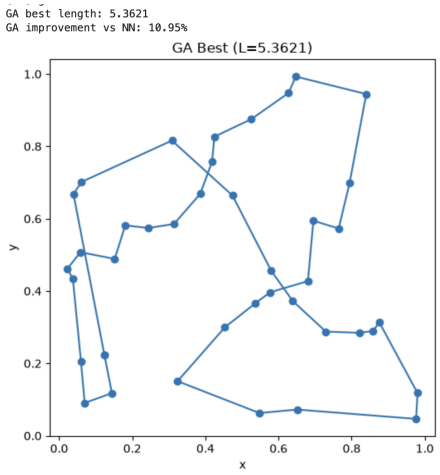
kept (elitism) and the nearest neighbor tour was put in the starting population and the rest were random.

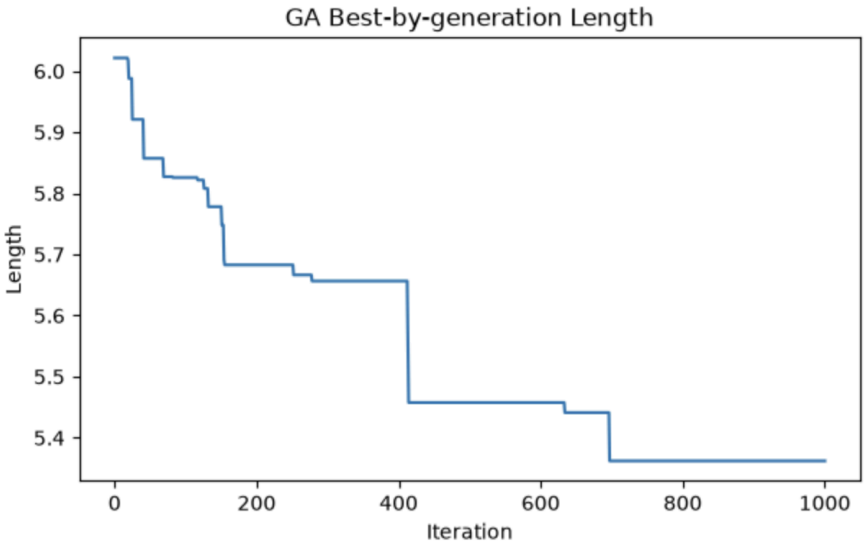
```
    # TODO: tournament selection (minimize length)
    contestants = rng.sample(range(len(pop)), k)
    winner = min(contestants, key=lambda i: lengths[i])
    return pop[winner][:]
```

```
    # TODO: order crossover (OX)
    n = len(parent1)
    a, b = sorted(rng.sample(range(n), 2))
    child = [None] * n
    child[a:b + 1] = parent1[a:b + 1]          # copy a slice from parent 1
    kept = set(parent1[a:b + 1])
    fill = [c for c in parent2 if c not in kept]  # remaining cities in parent 2's order
    j = 0
    for i in range(n):
        if child[i] is None:
            child[i] = fill[j]
            j += 1
    return child
```

```
    # TODO: swap mutation
    new_tour = tour[:]
    i, j = rng.sample(range(len(new_tour)), 2)
    new_tour[i], new_tour[j] = new_tour[j], new_tour[i]
    return new_tour
```

Parameters: population 500, 1000 generations, tournament size 3, crossover rate 0.9, mutation rate 0.5, elite size 2, seed 999.
Best tour length: 5.3621 (10.95% better than nearest neighbor).



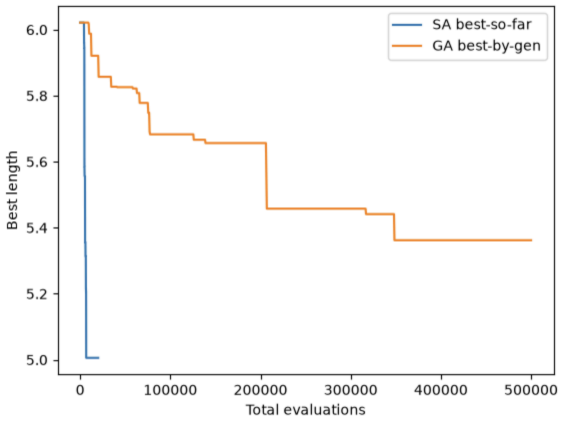


4. Results

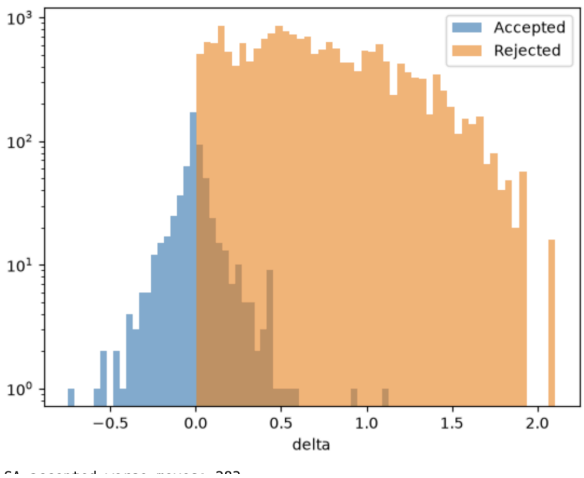
Method	Best length	Improvement vs NN
Nearest neighbor	6.0214	-
SA	5.0057	16.87%
GA	5.3621	10.95%

SA vs GA comparison plot,

```
NN: 6.0214 | SA: 5.0057 | GA: 5.3621
```

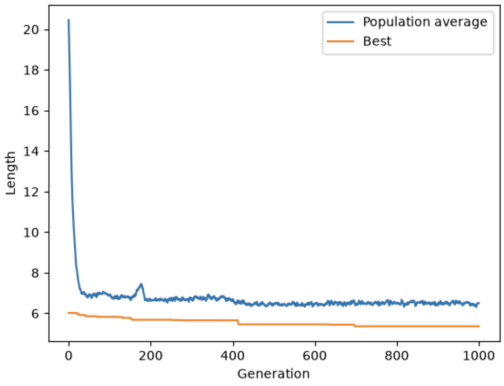


the delta histogram

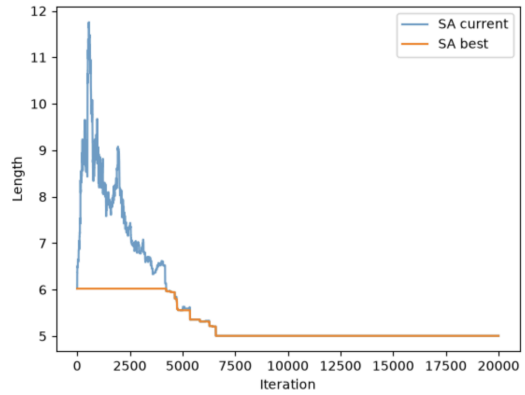


population average plot

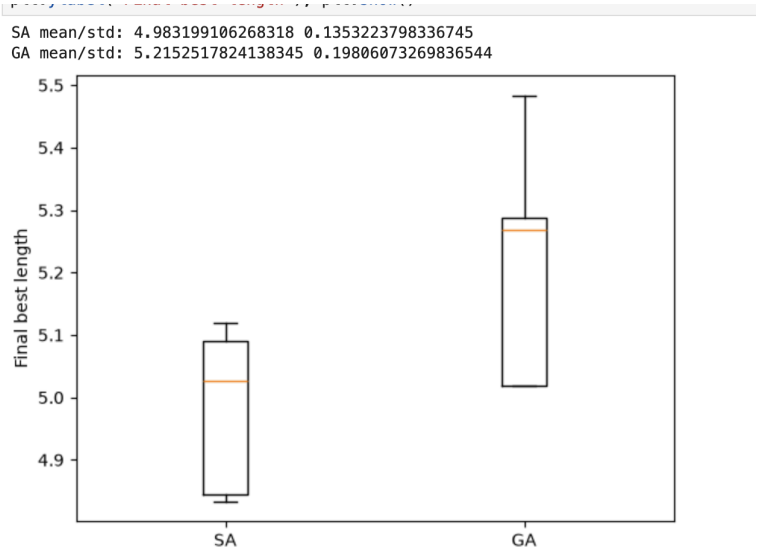
```
SA accepted worse moves: 283
```



SA current vs best plot



Boxplot



5. Analysis(Included in part 6 analysis in jupyter notebook)

- What algorithm converged faster? As you can see, SA converged a lot faster, reaching its best tour at 5.0057 after around 6,500 evaluations. GA needed around 350k evaluations(generation 700 with population 500) in order to reach its best tour at 5.3621. SA had already beaten GA's final result after right about 5,500 evals therefore SA is significantly more efficient for this problem.
- What showed more variability? GA showed more variability, across five different seeds SA finished with a mean of 4.98 and a standard dev of 0.14 while GA finished with a mean of 5.22 and a std dev of 0.20. The boxplot above shows the same thing, the GA results are actually more spread out and higher than the SA results. This is only 5 seeds, but even with it being a small sample, there's a clear trend.

- Did SA escape local minima and if so what Evidence? Yes, SA escaped local minima. It accepted 283 moves that made the tour longer. In the SA current vs best" plot, the current length of the tour increased from 6 to around 12 pretty early on(while temps were high) instead of getting stuck. Then it slowly came back down as the temp cooled off. The best tour only improved after that and reached 5.0057 at around 6,500. This shows SA was willing to go uphill to find a better area. After cooling it stayed for the best tour.
- Did GA lose diversity and if so what Evidence? Yes GA partially lost diversity. The number of unique tours in the population dropped from 500 at the start to 338 at the end, so about 1/3 of the population ended up becoming copies of other tours. GA's best length also stopped improving for long stretches(ex. it stayed at 5.3621 from generation 700 to 1000). This suggests the pop became too similar, which made it hard to find new improvements.
- Overall comparison :SA gave the better tour(5.0057 vs 5.3621) and was faster and more consistent. SA works well here because a 2-opt move ends up directly fixing the problem of crossing paths in a tour, so each step is a useful change. GA's swap mutation and crossover actually changes tours more randomly. This often breaks good parts of the tour, so it therefore improves more slowly. Both methods end up beating the nearest-neighbor baseline at 6.0214 by more than 10%(SA by 16.87% and GA by 10.95%).

6. Conclusion

Both methods(SA and GA) beat nearest neighbors by more than 10%, but SA ended up doing better overall. It not only found a shorter tour(5.0057 vs 5.3621), and got there with far fewer evaluations but also had a lower spread across seeds. I think that this is because 2-opt moves directly fix the crossing paths problem, while GA’s crossover and swap mutation change tours more randomly. A possible next step would be trying a different mutation for the GA, like reversing a section instead of swapping two cities.